Minimax on Nine Men's Morris

Nicholas Gazzo, 4498892

nicholasgazzo.gazzo@gmail.com

1: The project

The purpose of the project was to develop an agent which plays the game "Nine Men's Morris". As described in the proposal the 3 main aims of the project where:

1. Develop an engine for the game core functionalities.
2. Develop a graphical user interface to let the user interact with the game and partially with the agent settings.
3. Develop an agent for the interaction with the graphical user interface and one or more agents to evaluate the board state and decide which action to perform.

In this report I will explore in detail the design and development of all these parts while providing a basic instruction manual to use to software.

2: The game

Since there exists some variants of this game, I will briefly cover the basic rules, more information are available on the Wikipedia page.

The game is played on a board composed of twenty-four locations, usually called "*points*", connected by lines which determine the directions each piece can take to move. Each player has nine pieces, or "*men*", coloured black and white. The purpose of the game is to form "*mills*", three pieces of the same colour lined horizontally or vertically, that allow the player to remove one of the opponent's pieces. A player that has only two pieces left or that is unable to make a move lose the game.

The game starts within the "**placing phase**", where each player takes turns placing their pieces on empty points. If one of the players form a mill while placing his pieces, he can remove one of their opponent's pieces currently on the board (apart from pieces that are composing a mill).

After all pieces have been placed the game moves to the "**moving phase**", where players take turns moving a piece to an adjacent point. A piece can only move to an empty point and is not able to "*jump*" another piece. As before forming a mill while moving pieces lead to the removal of one of the opponent's pieces. When a player is reduced to three pieces, he has no longer limitation on the movement and can move his pieces on any empty point on the board.

3: Game engine

All the classes, interfaces and enumerator regarding the game engine are collected under the package "model". The class that incorporates all the core functionalities is NineMensMorris, an object of that class represents a current match in progress. The board is recreated with a 7x7 matrix unique for each object. Instead, the coordinates of all the points and their relationships of adjacency are stored inside static structures

available for all classes (GameConst). This kind of approach helps mitigate the redundancy of information (and consequent waste of memory) that other data structure would create during the building of multiple instances.

The matrix that represents the board is composed by objects of the class GamePiece, which is an Enum that encodes all the possible state of a point: occupied by a *Black Piece*, occupied by a *White Piece* or *Empty*. The "*turn*" variable, also of type GamePiece, is used to determine which player is supposed to act, where the "*phase*" variable instead establish what type of actions can be done by the player: *Placing Pieces*, *Moving Pieces*, *Removing Pieces* or *Game Ended*.
To perform an action from a phase it is sufficient to call one of the implemented methods, like: *movePiece*, *removePiece* or *addPiece*. Each of these methods return true if the action was performed correctly, false otherwise (for example if it wasn't a legal action in that phase).

Lastly, the class also implements the *Observer Java pattern* being the *observable* object which notifies all the *observer* whenever there is an important change in the game state, like: a turn change, a phase change or a game action (place/move/remove a piece).

4: Minimax algorithm

For this part I will not go into the details on how the algorithm works, since it was part of the course, but I will only focus on the aspects that I implemented and the choices that I had to make for this game.

4.1: Game tree

Usually, the game tree used in the minimax algorithm follows a scheme where at each level the nodes of that level represent all the possible actions that can be performed by a player. In a turn-based game usually the level of the game tree alternates between the players, however for this game there is a special case that we need to consider before moving on.

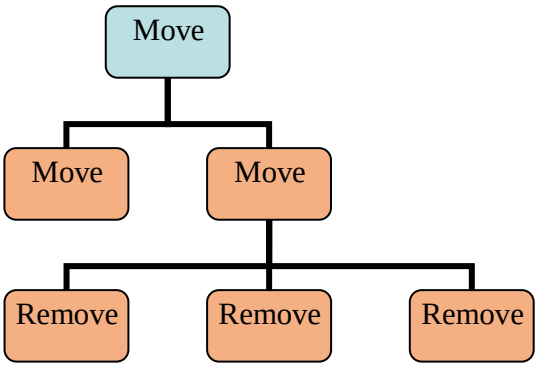

Figure 1 – In blue the maximizing player and red the minimizing player.

I'm talking about an action that create a mill, which can be both placing or moving a piece, when this situation occurs the consequent action is performed by the same player instead of switching it (Figure 1).

To solve this, I decided to consider the Move and Remove action as a single node, by creating as many Move nodes as the number of children Remove nodes (Figure 2).

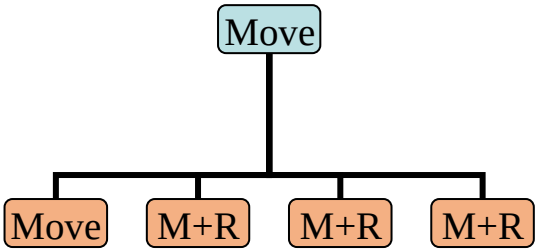

Figure 2 - Modified game tree

With this solution the game tree height is always consistent, meaning that a game tree of height *n* will always cover the next *n* turns. Also, with this solution the value of the

nodes takes in consideration the final board state, providing a more accurate view of how good the action was.

4.2: Data structure

Since the minimax algorithm works on a tree structure, I designed a class that could represents a node of my game tree, while trying to keep only the essential information for each node:

- List of child nodes
- A boolean, to represent if the node belongs to the maximizing player or the minimizing player.
- The board state, stored with a string format. This is needed to traverse the tree and move to the new root each time the opponent make a move.
- The board action that leads to the board state stored in the node. This is the information that we need to retrieve each time we apply the minimax algorithm.
- A value, that encode how good or bad the current board state is for the players. This is the value that the players try to maximize or minimize.

Note: The value information is only needed in the leaf nodes, and so is only computed for them. This helps to reduce the number of computations done during the building of the game tree.

4.3: Board state string format

Using a string to store the current board state in the tree structure instead of the whole object (NineMensMorris) helps mitigates the memory usage, which is important since the game tree will grow really fast. I tried to research if there was an existing standard to store this type of information (like there is for: chess, checkers and other board games) but I couldn't find any, so I decided to implement one that could fit my needs.

Here an example:

```
WWBBWWEWEBBEEBWEWEEBBWBE MB 18
```

The first twenty-four characters are used to represent a point on the board (starting from the top left corner), the 3 possible values are: *W* (White Piece), *B* (Black Piece) and *E* (Empty). The next two characters are used to encode the current phase and the current active player. For the phase we have: *P* (Placing Pieces), *M* (Moving Pieces), *R* (Removing Pieces) and *E* (Game Ended). Finally, the last characters are used to keep track of how many pieces the players have put on the board (which is needed only during the placing phase).

4.4: Evaluating the board

The key element of the minimax algorithm is being able to accurately evaluate the current game state, since all the decision that the agent will make are based on minimizing or maximizing this value. This is not a trivial task; finding a good evaluation method usually requires a lot of experiments and a lot of time.

Since I'm not an expert on this field I tried to research a bit for articles discussing this topic, unfortunately most of these articles are related to other (more complex) board

games. There were some articles about implementing the minimax algorithm on the Nine Men's Morris, but they were more focused on discussing how the minimax algorithm works and not the method used to evaluate the board state.

So, I tried to design a basic algorithm to evaluate the board state using my knowledge of the game and following some simple concepts:

1. If the game has ended, then we assign some fixed values: 0 to the losing player and 999 to the winning player.
2. A player with more pieces has more chances of winning.
3. If we are in the placing phase, we also need to consider the number of pieces that we can still place on the board and not only the ones that are currently on the board (since there could be ambiguous case where both players have the same number of pieces on the board but not the same number of pieces in hand).
4. We need to value more the pieces that are currently in a mill compared to pieces that are not (since they cannot be removed, and they can move back and forth to make a mill).
5. We need to consider the number of moves that a player has at a given state (since a player that is not able to make a move lose).

After assigning a weight to each of the information mentioned (pieces on the board, pieces in hand, pieces in a mill and number of moves) we sum them up and obtain a positive value that represent how good the board state is for a certain player.

$$Value(Player) = \begin{cases} 0 & (if\ the\ game\ ended\ and\ the\ Player\ lost) \\ 999 & (if\ the\ game\ ended\ and\ the\ Player\ won) \\ W_1 * Pieces_{Board} + W_2 * Pieces_{Hand} + W_3 * Pieces_{Mills} + W_4 * Moves \end{cases}$$

Finally, to assign the value at each leaf node of the game tree, we evaluate the difference between the value of the board for the maximizing player and value of the board for the minimizing player.

$$Value(Node) = Value(Player_{Maximizing}) - Value(Player_{Minimizing})$$

Note: I decided to consider the Agent as the maximizing player and the user as the minimizing one.

Obviously, this is not a refined algorithm at all, it would require more experimentation on the chosen weights and more game knowledge to identify some missing elements. Still, the algorithm can provide some good results (especially during the placing phase) depending on the depth of game tree.

4.5: Alpha & Beta pruning

Since the large number of actions that can be performed at each turn, the game tree tends to grow really fast, making the traverse of the tree slow. In order to speed up the computations of the optimal move, I also applied the alpha & beta pruning, an improvement of the minimax algorithm that seeks to decrease the number of nodes that are evaluated. This can be done thanks to the use of two variables:

- Alpha: which stores the minimum score of the maximizing player.
- Beta: which stores the maximum score of the minimizing player.

By keeping track of these values at each iteration, the algorithm can stop evaluating a branch of moves when at least one better solution has already been found.

5: Agents

The project implements three different agents, one which interact with the user interface, one which controls all the operations regarding the game tree and minimax algorithm and one that function as a logger for the other agents.

5.1: GUI Agent

The GUI Agent controls all the interaction between the agents and the graphical user interface. The constructor of the object takes as argument the reference to the GUI that will be used in order to call for its methods when needed. This agent doesn't have any active behaviour when it's created, all the behaviour executed by the agent are added only when a change occurs in the environment (the GUI).

When a change occurs in the environment (for example when the turn passes from the player to the agent) the GUI will call a method on the agent that will add a **OneShotBehaviour**, which is a behaviour that execute only once. Depending on the method called the operations done in the behaviour are different:

- Turn Changed: the agent prepares a message of type *Request* for the Minimax Agent, putting the game state of the board (string format) as content for the message. Then after sending the message, it will wait for a response with a blocking receive. After receiving an answer which should be an object of type **BoardAction**, the agent will perform that action calling the corresponding method of the GUI.
- Settings Changed: the agent prepares a message of type *Inform* for the Minimax Agent, putting the changed value in the settings as content. This time the agent will not wait for a response.

In both the behaviours the agent will also try to log useful information by sending messages of type *Inform* or *Failure* to the Logger Agent. For this agent I decided to use **OneShotBehaviour** instead of **CyclicBehaviour** because they are better suited to handle this event driven scenario like this one, since they are added and executed only when needed.

5.2: Minimax Agent

The Minimax Agent controls all the operations regarding the game tree and minimax algorithm. The agent implements a single **CyclicBehaviour** that keep listening until a it receives a message that match its MessageTemplate (in this case messages of type *Inform* or *Request*). When the agent receives a message that match the template it will act differently depending on the type of the message:

- Inform: the agent will retrieve the content of the message; it will try to cast it to integer and then it will use it as the new max depth for the future game trees.
- Request: the agent will retrieve the content of the message, which should be a board state (represented with string format). Then, if the game tree has already been constructed or not it will try to move to the new root (using the retrieved board state). If it's not possible to find the new root node (in the children of the current root node) or if the tree has not been constructed yet, then the agent

instantiates a new root node using the retrieved board state. Once the root node has been established then the agent will generate the children node up to the max depth parameter (one child for each possible action from that board state).

At this point it will be added a **OneShotBehaviour** to the agent that will handle the response to the GUI Agent, this behaviour will only be executed once the game tree has generated all the child nodes. During this behaviour the agent will use the minimax algorithm to retrieve the optimal move in the leaf nodes. Then, once the optimal value is calculated the agent will retrieve the BoardAction that lead to that branch of moves, and it will send it to the GUI Agent. And it will set that node as the new root node (that will be used in the next request).

As the other agent in both the behaviours the agent will try to log useful information by sending messages to the Logger Agent. For this agent I also considered the use of **ParallalelBehaviour**, to use them inside the computation of the optimal move with the minimax algorithm. I ultimately discarded the idea since it would require synchronized data structures to store alpha and beta (to avoid race condition) and it could also harm the amount of pruning done by the algorithm, since it would depend on the execution order of the behaviours.

5.3: Logger Agent

The Logger Agent is the simpler agent among the one that I implemented for this project. Its constructor takes a **LoggerOutput** as argument, which is an Interface that ask to implement only two methods: *logMessage* and *logError*. It consists only of one **CyclicBehaviour** that keep listening waiting for messages of type *Inform* or *Failure* (using a MessageTemplate to filter only these type of messages), when it receives one of these types of messages it calls the corresponding method of the logger output.

For the purpose of this project the LoggerOutput interface is implemented by the graphical user interface which just log the messages inside a Text Area. The intended use of this class would be to use this agent to log inside a database or a text file, depending on the needs of the application.

5.4: Swing and JADE

Jade provides different thread of execution for each agent, but since I already used Swing, I know for experience that updating the GUI from within this thread can lead to unexpected problems due to synchronization. So, to avoid this problem I followed the recommended practices explained in the “Developing Multi-Agent Systems with Jade” book. The recommended practice is to encapsulate all the accesses to the GUI from the Jade behaviours with Runnable, and then submit them to the EventDispatchThread using the InvokeLater method.

Another recommended practice is to avoid reacting to an AWT Event by blocking the event dispatcher (waiting for the response of an ACL Message), instead the optimal solution would be to incorporate the operations inside behaviours that will be later scheduled for execution by the agent’s thread.

6: The Graphical User Interface

This section will aim mostly at providing an overview of the implementation of the interface as well as a guide to use it. The Graphical User Interface has been developed using the *Swing Java library*, with the help of the *Window Builder Tool* provided by the Eclipse IDE.

6.1: How to use the GUI

The window application is divided into tree tabs:

The first one, which is also the starting one is the **Menu**. From this tab it is possible to start a new game, save the one that is currently going or load a previously saved game. Whenever the user tries to start a new game o load a saved one, if there is already a game in progress, the user will be prompted with a window that ask for confirmation of the action. After the confirmation the tab will be switched to Game. By default, the game will always choose the user as the first player assigning him the *White Pieces*, the agent instead will go second and will be assigned the *Black Pieces*.

Figure 4 – The Menu tab

The second tab is **Game**, where it is possible to interact with the game board and perform actions. The labels on the top part of the window, show to the user what is the current phase and who is the active player. The user can only interact with the board during his turn.

During the *Placing Pieces Phase*, in order to put a piece on the board it is sufficient to click on an empty point. To move a piece during the *Moving Pieces Phase* instead, firstly it needs to be selected by clicking on it. Then the selected piece will be highlight with an orange hue, and it will be possible to move it to another position by clicking on an adjacent empty spot. Lastly, during the *Removing Pieces Phase* to remove an opposing piece it is sufficient to click on it.

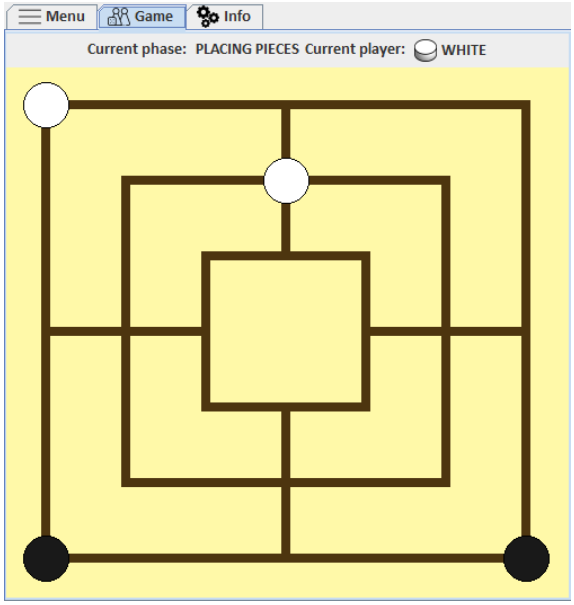

Figure 5 – The Game tab

The last tab is **Info**, from here it is possible to look at a recap of all the actions performed during the current game and to interact with the Agent by changing the tree depth.

To change the tree depth, it is sufficient to change the value of the corresponding spinner. The available values are contained in the range that goes from one to ten. The bottom text area instead shows all the messages received by the *Logger Agent*, writing for each of them the agent who sent it and its content.

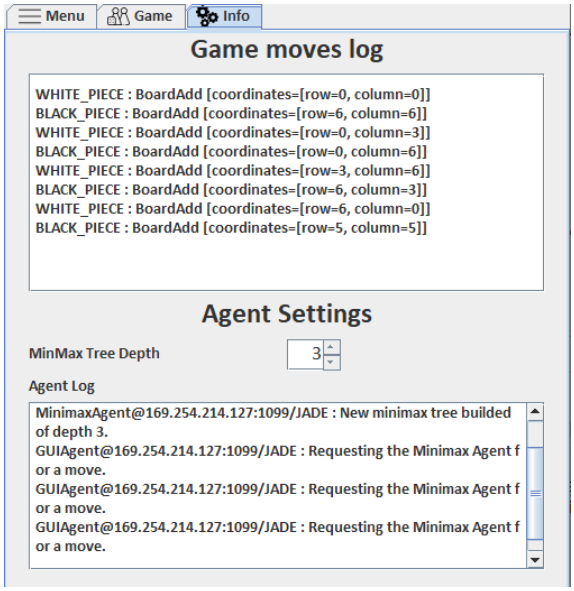

Figure 6 – The Info tab

6.2: Notes on the implementation

The GUI implements the BoardListener interface which helps to manage all the updates whenever an important event occurs, such as a turn change, a phase change or a board action. This is also used to log the moves information inside the Text Area in the Info Tab and to trigger the request of a new move by the GUI Agent.

The graphical elements of the game are drawn with the use of a custom class called NineMensMorrisCanvas, which keep a reference at the game engine object and call the repaint method each time there is a relevant interaction with the user or the agent.

7: The Results

I'm overall happy with the results obtained, the agent seems to work better during the placing phase (where it has more freedom on the position of the pieces) and it seems to struggle more during the moving phase, especially when it should close the game (winning).

I tried to use different levels of depth for the game tree, but I couldn't go past depth 5 (without any other application running) before running out of memory. The best configuration that I tested was with a max depth of 3 (that I selected as the default one), where the agent seems to be "smart" enough to punish a player which plays without a reasonable plan (by closing mills and eating the opponent's pieces).

References

[1] Fabio Bellifemine (Telecom Italia, Italy), Giovanni Caire (Telecom Italia, Italy), Dominic Greenwood (Whitestein Technologies AG, Switzerland). Developing Multi-Agent Systems with JADE, 2007.

[2] Evan Hurwitz and Tshilidzi Marwala. Multi-Agent Modeling Using Intelligent Agents in the Game of Lerpa.